

UNIVERSIDAD INTERNACIONAL DEL ECUADOR

Programación Orientada a Objetos 2



LeeLibre

Planeación de un Sistema de Gestión de Libros Electrónicos

Desarrollado bajo Programación Funcional en Go

Alumno: Cortez Villa Ronny

Docente: Ing. Torres Arévalo Carlos

Etapas 1 — Planeación del Software

17 de julio de 2026

Resumen

El presente documento corresponde a la Etapa 1 del proyecto integrador de la unidad, cuyo objetivo es planear el desarrollo de *LeeLibre*, un sistema de gestión de libros electrónicos que permitirá a una comunidad de lectores consultar un catálogo, registrar préstamos digitales y llevar un historial de lectura. La planeación se sustenta en el paradigma de programación funcional, implementado en el lenguaje Go a través de funciones de primera clase, funciones de orden superior y estructuras inmutables. Se describen el alcance del proyecto, los tres módulos principales (catálogo, usuarios y préstamos) junto con un módulo transversal de reportes, la arquitectura por capas propuesta, el modelo de datos, los paquetes de terceros seleccionados y el cronograma de trabajo distribuido en tres etapas. El documento se acompaña de seis representaciones gráficas (casos de uso, arquitectura, módulos, entidad-relación, flujo del proceso principal y cronograma de Gantt) que sirven como base para las etapas posteriores de desarrollo e integración.

Palabras clave: programación funcional, Go, gestión de biblioteca digital, arquitectura por capas, planeación de software.

Introducción

En los últimos años el consumo de libros digitales ha crecido de manera sostenida, tanto por la comodidad de acceder a un catálogo desde cualquier dispositivo como porque muchas instituciones educativas migraron parte de su bibliografía a formatos electrónicos. Ese contexto abre la posibilidad de construir sistemas que administren esa oferta de forma ordenada, es decir, aplicaciones que sepan quién presta qué, durante cuánto tiempo y con qué historial. Ese es el problema que este proyecto propone atender mediante *LeeLibre*, una aplicación web ligera para gestionar una biblioteca digital.

Esta primera entrega no se ocupa del código ni del diseño visual, sino de dejar claro *qué* se va a construir, *cómo* se va a organizar y *con qué* se va a construir. El documento sigue la estructura solicitada: objetivos y alcance, justificación de la elección del sistema, marco referencial con énfasis en el paradigma funcional (requisito explícito del enunciado), descripción de módulos, arquitectura, modelo de datos, paquetes, cronograma y configuración del repositorio en GitHub.

Objetivos

Objetivo general

Planear el desarrollo de un sistema de gestión de libros electrónicos, denominado LeeLibre, aplicando los principios de la programación funcional y una arquitectura por capas, de modo que las etapas posteriores del proyecto integrador cuenten con una base técnica clara y documentada.

Objetivos específicos

1. Definir el alcance funcional del sistema, identificando los actores, los módulos principales y las operaciones que cada módulo debe soportar.
2. Diseñar la arquitectura de la aplicación en capas y representarla mediante diagramas de casos de uso, de componentes, entidad-relación y de flujo, con el fin de comunicar de manera visual la estructura del software.

3. Justificar la selección del lenguaje Go y de los paquetes de terceros a utilizar, describiendo cómo cada uno contribuye al enfoque funcional y a la calidad del producto.
4. Establecer un cronograma de trabajo dividido en las tres etapas del proyecto integrador y configurar un repositorio en GitHub que permita versionar el código de manera ordenada.

Alcance del Proyecto

LeeLibre es una aplicación web para uso interno de una biblioteca digital. En su versión final permitirá consultar un catálogo filtrando por categoría o autor, registrar el préstamo digital de un libro por parte de un usuario autenticado, y consultar el historial personal de lectura. También incluirá un módulo administrativo básico que permita a un usuario con rol de *administrador* agregar, modificar o retirar libros del catálogo.

Por tratarse de un proyecto individual desarrollado a lo largo de tres etapas, el alcance se acota. **No** formarán parte de esta versión: la integración con pasarelas de pago, un visor de libros dentro del navegador, un sistema de recomendaciones con aprendizaje automático, ni el soporte para múltiples idiomas. Se probará con un conjunto pequeño de datos ficticios (aproximadamente treinta libros y diez usuarios), suficiente para demostrar las funcionalidades en el video de sustentación de la Etapa 3.

Justificación de la Elección del Sistema

El enunciado permite elegir entre tres opciones: un sistema de gestión de libros electrónicos, uno de streaming y uno de comercio electrónico. Elegí el sistema de libros electrónicos por tres razones.

Primero, por el dominio: la lógica de una biblioteca digital se entiende con facilidad y no exige conocimiento especializado (a diferencia, por ejemplo, del manejo de codificadores de video en streaming o de las pasarelas de pago en un comercio electrónico). En una entrega individual esto es importante porque permite dedicar tiempo a la calidad del código y no a

resolver problemas ajenos al curso.

Segundo, por el encaje con el paradigma funcional. Buena parte de las operaciones de un catálogo (filtrar por categoría, transformar registros, calcular estadísticas, ordenar por popularidad) son transformaciones sobre listas, y ese es el terreno donde la programación funcional muestra su valor. Un comercio electrónico, en cambio, gira alrededor de un carrito de compras con estado mutable, incómodo de modelar en un enfoque funcional puro.

Tercero, por la sostenibilidad del proyecto. Como el mismo proyecto se llevará hasta la tercera entrega, conviene elegir un sistema cuya complejidad pueda crecer de forma controlada. El catálogo puede empezar con operaciones simples e incorporar reportes o recomendaciones sencillas sin cambiar la estructura del código.

Marco Referencial

Programación funcional

La programación funcional es un paradigma que trata a las funciones como el elemento central del programa. En un enfoque funcional puro las funciones se comportan como en las matemáticas: reciben una entrada y devuelven una salida sin modificar nada externo, propiedad conocida como *transparencia referencial*. Los conceptos que se aplicarán a lo largo del proyecto son cuatro: (1) **funciones de primera clase**, que pueden asignarse a variables y pasarse como argumentos; (2) **funciones de orden superior**, que reciben o devuelven otras funciones y se usarán para construir filtros sobre el catálogo; (3) **inmutabilidad**, donde los datos no se modifican sino que se producen copias con la modificación; y (4) **composición**, donde funciones pequeñas se combinan para formar operaciones más complejas.

Go como lenguaje de implementación

Go es un lenguaje multiparadigma diseñado por Google. No es un lenguaje funcional puro (como Haskell o Elm), pero soporta los cuatro conceptos mencionados: permite pasar funciones como argumentos, definir closures, retornar funciones desde otras funciones y construir estructuras inmutables por convención mediante el uso disciplinado de **struct** sin

métodos que muten sus campos.

La decisión de usar Go se apoya en tres criterios: la curva de aprendizaje es accesible para un estudiante de tercer semestre; existen librerías estables para desarrollo web (Gin), acceso a bases de datos (`database/sql`) y pruebas (`testing`); y el despliegue es sencillo porque un programa en Go se compila a un binario único, lo que facilita la demostración en el video final. El costo es que el paradigma funcional se aplica por disciplina y no por obligación del lenguaje; esta convención se documentará en el repositorio. Como ilustración del estilo funcional, la siguiente función de orden superior filtra libros a partir de un predicado:

```
// Filter recibe una lista de libros y un predicado,  
// y devuelve una nueva lista sin modificar la original.  
func Filter(libros []Libro, pred func(Libro) bool) []Libro {  
    resultado := make([]Libro, 0, len(libros))  
    for _, l := range libros {  
        if pred(l) {  
            resultado = append(resultado, l)  
        }  
    }  
    return resultado  
}
```

Arquitectura por capas

El sistema se organizará en tres capas: presentación, lógica de negocio y datos. Esta separación permite modificar una capa sin afectar a las otras (por ejemplo, cambiar la base de datos sin tocar la lógica) y hace explícito qué código pertenece a cada preocupación, lo que mejora la lectura del proyecto por parte del docente al revisar el repositorio.

Descripción del Sistema y sus Módulos

LeeLibre se compone de tres módulos principales y un módulo transversal de reportes. A continuación se describen indicando su responsabilidad y su encaje con el enfoque funcional.

Módulo de catálogo

Es el núcleo del sistema. Administra la información de los libros: título, autor, categoría, año, formato (PDF o EPUB) y disponibilidad. Sus operaciones son CRUD sobre los libros y búsquedas por título, autor o categoría. Las búsquedas se implementan como funciones de orden superior que reciben un predicado y devuelven la lista filtrada, sin modificar el catálogo original.

Módulo de usuarios y autenticación

Administra el registro y el ingreso de usuarios. Cada usuario tiene un rol asignado (*lector* o *administrador*). También se ocupa de la sesión mediante *tokens* JWT y del cifrado de contraseñas con *bcrypt*. Aunque el manejo de sesiones es un caso donde el estado es inevitable, se aísla en una capa específica para que el resto del sistema pueda mantenerse sin estado.

Módulo de préstamos digitales

Registra el préstamo de un libro por parte de un usuario. Al ser un préstamo digital, representa el permiso para descargar el libro por un periodo determinado (catorce días). Lleva el historial de préstamos por usuario y calcula el estado actual de cada préstamo (vigente, devuelto o vencido). Estas transiciones se modelan como funciones puras que reciben el préstamo y una fecha, y devuelven el nuevo estado sin alterar el préstamo anterior.

Módulo transversal de reportes

Consulta los datos generados por los tres módulos anteriores y produce vistas agregadas: libros más prestados, categorías más consultadas y usuarios más activos. Se implementa exclusivamente con operaciones funcionales sobre las listas de datos existentes (*map*, *filter*, *reduce*), lo que refuerza el paradigma en la parte del sistema donde más natural resulta.

Arquitectura y Representaciones Gráficas

Esta sección presenta las seis representaciones gráficas que sustentan la planeación. Cada diagrama se acompaña de una explicación breve sobre qué comunica y por qué se incluyó.

Diagrama de casos de uso

El diagrama de la Figura 1 muestra los dos actores del sistema (lector y administrador) y las principales acciones que cada uno puede realizar. Sirve para acotar el alcance funcional del sistema en un único gráfico y para verificar que ningún módulo quedó sin usuario asignado.

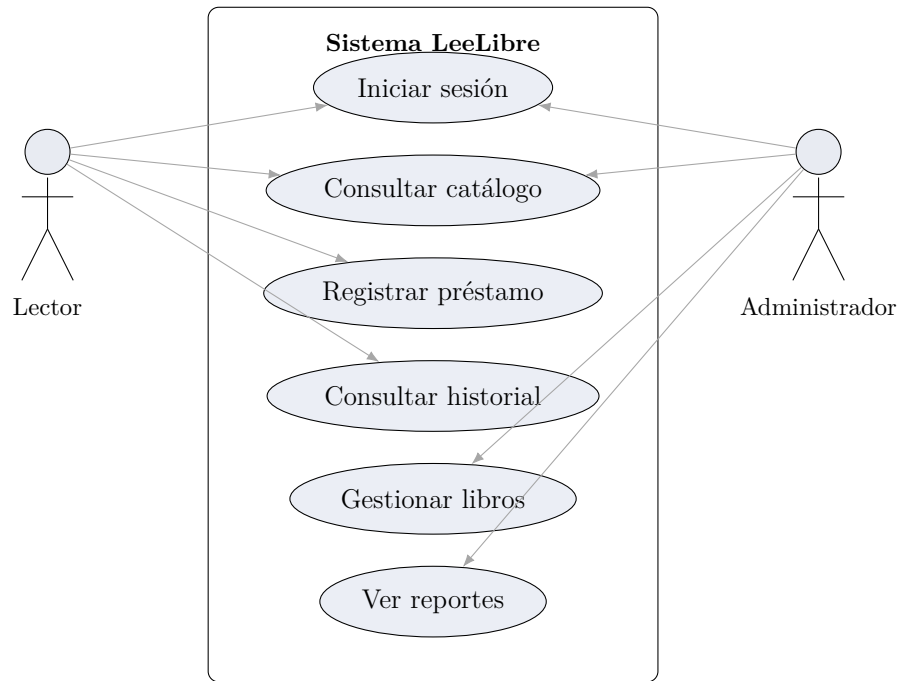


Figura 1. Diagrama de casos de uso de LeeLibre.

Del diagrama se desprende una decisión de diseño: el administrador no registra préstamos, esa acción queda del lado del lector. Con eso se evita el sesgo de un administrador que use el sistema como si fuera un lector encubierto.

Arquitectura por capas

La arquitectura de la Figura 2 separa el sistema en tres capas más una base de datos. La capa de presentación recibe las peticiones HTTP y devuelve HTML o JSON. La capa de negocio contiene las funciones puras que operan sobre los datos. La capa de datos abstrae el acceso a SQLite mediante repositorios.

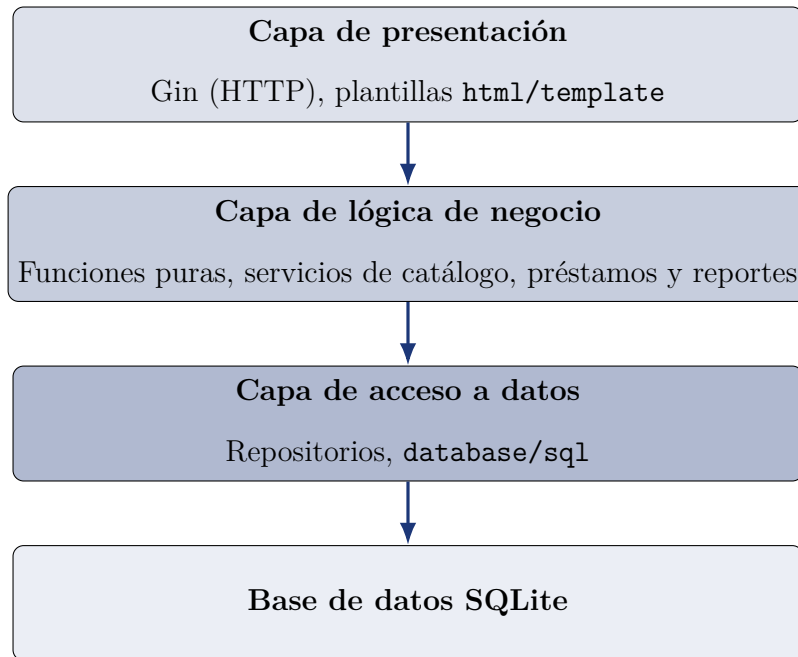


Figura 2. Arquitectura por capas del sistema LeeLibre.

La ventaja de este orden es que si en el futuro se decide cambiar SQLite por PostgreSQL, solo se toca la capa de datos. La capa de negocio se mantiene intacta porque solo depende de las interfaces de los repositorios.

Módulos y dependencias

La Figura 3 muestra cómo los tres módulos principales dependen unos de otros. El módulo de reportes es el único que consulta a los tres, lo cual es esperable dado que su función es agregar información.

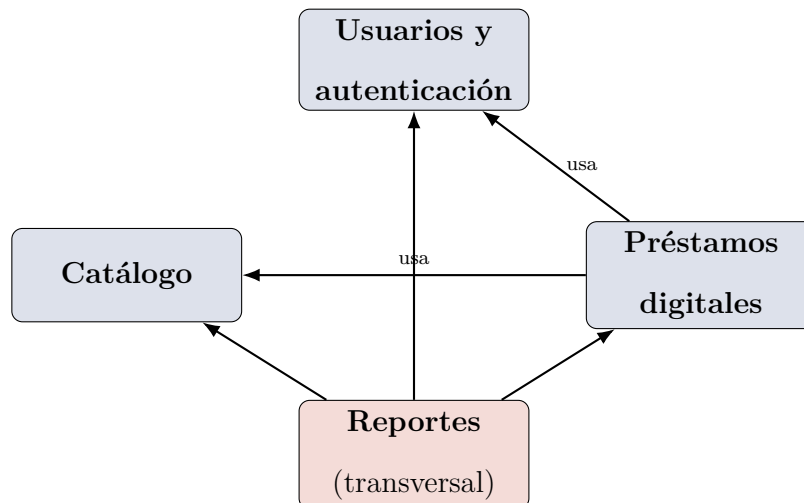
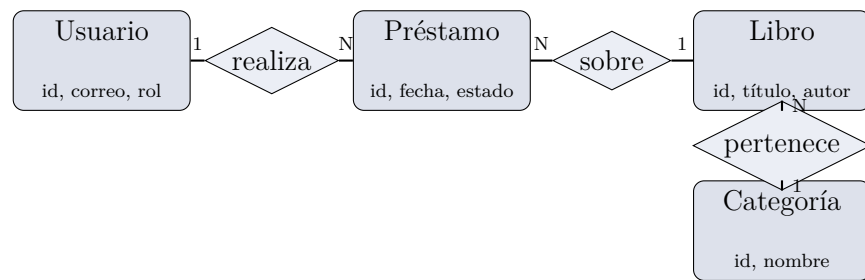


Figura 3. Módulos del sistema y sus dependencias.

Un principio que se mantendrá durante el desarrollo es que el módulo de catálogo no debe depender de préstamos ni de usuarios: es el módulo más básico y debe poderse probar de manera aislada.

Modelo entidad-relación

La Figura 4 representa el modelo de datos. Se incluyeron únicamente las cuatro entidades necesarias para las funcionalidades del alcance: usuario, libro, categoría y préstamo.

*Figura 4.* Modelo entidad-relación de LeeLibre.

La entidad *Préstamo* funciona como tabla puente entre usuario y libro, con la particularidad de que guarda su propio estado. Esa decisión permite reconstruir el historial completo sin necesidad de auditorías adicionales.

Flujo del proceso principal

El flujo de la Figura 5 describe el recorrido que hace un lector desde que ingresa al sistema hasta que registra un préstamo. Es el escenario que se demostrará en el video de la Etapa 3.

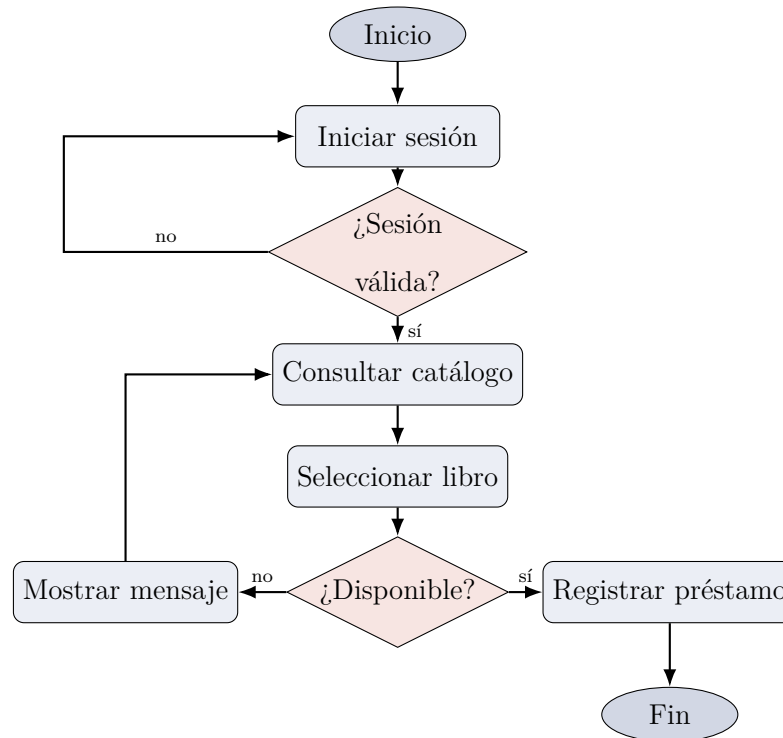


Figura 5. Flujo del proceso principal: consulta y préstamo de un libro.

Se incluyó una rama alternativa (libro no disponible) que devuelve al usuario al catálogo. Esa transición pequeña es la que garantiza que el sistema no llegue a un estado bloqueado.

Paquetes y Librerías

El proyecto combina la biblioteca estándar de Go con algunas librerías externas. Se documenta a continuación cada una, tal como lo exige el enunciado.

Paquete	Origen	Uso en el proyecto
<code>net/http</code>	Estándar	Base para el servidor web; utilizado indirectamente a través de Gin.
<code>database/sql</code>	Estándar	Interfaz genérica de acceso a la base de datos.
<code>html/template</code>	Estándar	Renderizado de las vistas HTML.
<code>testing</code>	Estándar	Pruebas unitarias de las funciones puras del módulo de catálogo y de préstamos.

Paquete	Origen	Uso en el proyecto
<code>gin-gonic/gin</code>	Tercero	Framework HTTP ligero. Se eligió por su bajo consumo de recursos y su compatibilidad con estilo funcional en el manejo de middlewares.
<code>mattn/go-sqlite3</code>	Tercero	Driver para SQLite, compatible con <code>database/sql</code> .
<code>golang-jwt/jwt</code>	Tercero	Emisión y verificación de <i>tokens</i> para sesiones.
<code>x/crypto/bcrypt</code>	Tercero (oficial)	Cifrado de contraseñas de usuarios.

Tabla 1

Paquetes seleccionados para el proyecto.

Todos los paquetes de terceros son de código abierto, están mantenidos activamente y su licencia (MIT o similar) permite su uso en proyectos académicos. La versión exacta de cada paquete se fijará en el archivo `go.mod` del repositorio.

Cronograma de Trabajo

El cronograma de la Figura 6 distribuye las actividades a lo largo de las tres etapas del proyecto integrador. Se estiman semanas de trabajo tomando en cuenta que se trata de una entrega individual.

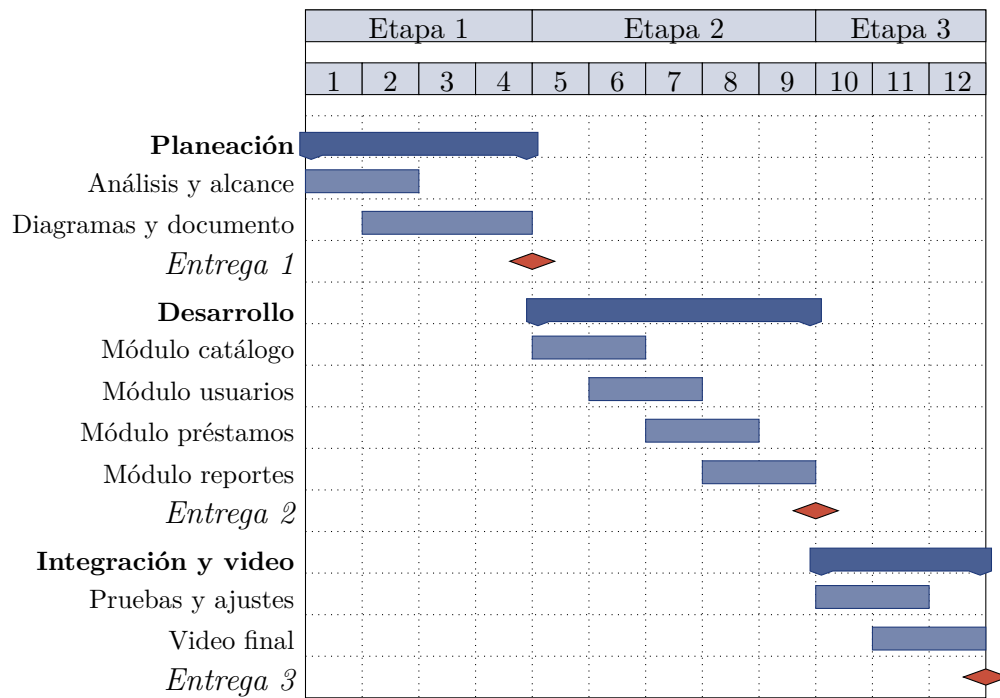


Figura 6. Cronograma general del proyecto (unidades en semanas).

Configuración del Repositorio en GitHub

El repositorio se aloja en GitHub bajo el nombre `leelibre` y sigue la convención habitual de un proyecto en Go:

```
leelibre/
|-- cmd/server/      # punto de entrada de la aplicación
|-- internal/
|   |-- catalogo/    # módulo de catálogo
|   |-- usuarios/    # módulo de usuarios
|   |-- prestamos/   # módulo de préstamos
|   |-- reportes/    # módulo transversal
|-- web/
|   |-- templates/   # vistas html/template
|   |-- static/      # css e imágenes
```

```
|-- migrations/          # esquemas SQL
|-- docs/                # documentación de las tres entregas
|-- go.mod, go.sum, README.md, LICENSE
```

Se utilizarán dos ramas principales: **main** para versiones estables al final de cada etapa y **develop** para el trabajo en curso. Cada módulo se desarrollará en una rama corta que se fusionará con **develop** mediante *pull request* interno; aunque se trata de un proyecto individual, esta práctica queda documentada para cumplir con el requisito de trabajo por versiones. El archivo **README.md** incluirá el propósito del proyecto, las instrucciones de instalación y el listado de dependencias.

Conclusiones Parciales de la Etapa 1

La planeación permitió pasar de una consigna abierta a una propuesta concreta con alcance, arquitectura y cronograma definidos. Del ejercicio surgen tres observaciones. Primero, la elección del sistema de gestión de libros por encima de las otras dos opciones no fue por preferencia personal sino por su encaje con el paradigma funcional: modelar transformaciones sobre listas es más didáctico en un catálogo que en un carrito de compras. Segundo, el uso de Go obliga a mantener la disciplina funcional por convención ya que el lenguaje no la impone, y esa disciplina se documentará en el repositorio para que quede evidenciada en la Etapa 2. Tercero, el cronograma es ajustado pero realista para una persona, y reserva margen para las pruebas y la elaboración del video. Con esta planeación entregada, la Etapa 2 puede comenzar directamente en la implementación del módulo de catálogo, que por su papel de núcleo debe estar listo antes que los demás.

Referencias

- [1] American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7^a ed.). <https://doi.org/10.1037/0000165-000>
- [2] Donovan, A. A. A., & Kernighan, B. W. (2015). *The Go programming language*. Addison-Wesley.

- [3] Hughes, J. (1989). Why functional programming matters. *The Computer Journal*, 32(2), 98–107. <https://doi.org/10.1093/comjnl/32.2.98>
- [4] Sommerville, I. (2016). *Software engineering* (10^a ed.). Pearson.
- [5] Fowler, M. (2018). *Refactoring: Improving the design of existing code* (2^a ed.). Addison-Wesley.
- [6] Gin Web Framework. (2024). *Gin documentation*. <https://gin-gonic.com/docs/>